

EDYGUIDE

Student Service Chatbot

AI-Powered Egyptian University Portal

Full Project Documentation

User Flow | UI Screens | Technology Stack | Core Code

Methodology | Database | Security | Roadmap

Chapter 1: Complete User Journey & Screen Flow

This chapter traces the full student experience inside EDYGUIDE — from the first time they open the app through every major service. Each screen is documented with its purpose, components, and the logic that drives it.

Step	Screen	Trigger	Next Action
1	Home Page (Welcome)	App launch	Login button
2	Login / Authentication	Click 'Get Started'	Verify credentials
3	Dashboard	Login success	Select service
4	Chatbot (AI Concierge)	Click 'Start Chat'	AI response loop
5	Academic Services	Click grade/schedule	View data
6	Payment Screen	Click 'Pay Tuition'	Fawry/Vodafone
7	Career Bridge	Click 'Career Advice'	Upload CV / Browse jobs
8	Wellness Hub	Click 'Mental Health'	Mood check / Chat Saleem
9	Progress & Nudge	Automated (SVM)	Push notification sent

Screen 1 Home Page — Welcome Screen

The first screen the student sees. Displays the university branding, EDYGUIDE logo, and a welcoming message. No login required yet — this is a public-facing landing page.

- University logo + EDYGUIDE owl icon in the header (navy blue bar)
- Tagline: 'Your Student Service Assistant — Available 24/7'
- Feature highlights: Instant Responses | Academic Help | Easy & Convenient
- Prominent 'Get Started' / 'Start Chat' CTA button (blue, centered)
- Brief description of what EDYGUIDE does
- Robot/chatbot illustration for visual appeal
- Navigation: Home | About | Contact (minimal — user not logged in yet)

Screen 2 Login & Authentication Screen

Student enters their university credentials. The system authenticates via the Student Information System (SIS/Banner) using SSO.

- Input fields: Student ID and Password
- University SSO (Single Sign-On) integration with Banner/LDAP
- Multi-Factor Authentication (MFA) — OTP sent to university email or phone
- 'Forgot Password?' link → redirects to IT support chatbot
- Error states: wrong credentials show red inline message
- On success → JWT token issued, user redirected to Dashboard
- Law 151/2020 privacy notice displayed before first login
- Session management: auto-logout after 30 minutes of inactivity

Screen 3

Main Dashboard — Personalized Home

After login, the student sees their personalized dashboard. This is the control center of EDYGUIDE, showing all services and proactive AI nudges based on real-time SIS data.

- Personalized greeting: 'Welcome Ahmed — How can I assist you today?'
- DAILY NUDGE CARD (gold banner): 'You have Physics at 10 AM — you have missed 2 classes, don't miss this one!'
- Service Grid (mobile 2x4, desktop 3x3):
 - Chat with Bot | Schedule | Exam Dates | FAQ
 - Pay Tuition | View GPA | Course Registration | Attendance
 - Career Advice | Mental Health Support
- Bottom navigation bar (mobile): Home | Chat | Payments | Career
- Top navigation bar (desktop): Home | Chat | Payments | Career | Search | Notifications | Profile
- Notification bell with unread count badge

Screen 4

AI Chatbot — Concierge Screen

The core feature. A conversational AI interface powered by Llama-3. Students ask any academic, administrative, or advisory question in Arabic or English.

- Chat bubble interface (blue = bot, white = student)
- Text input bar + microphone icon for voice input
- Quick reply chips: 'Exam Dates' | 'My Schedule' | 'GPA Prediction' | 'Pay Fee'
- Bot opens with: 'Hi! How can I assist you today?'
- Predictive GPA: 'What is my GPA if I get a B in Math?' → AI calculates and replies
- Arabic dialect support (MSA + Egyptian colloquial)
- Typing indicator (animated dots) while AI processes
- Session history — scrollable chat log
- Escalation button: 'Talk to a Human' → creates admin ticket
- File attachment: student can upload documents for processing

Screen 5 Academic Services — Grades, Schedule & Attendance

Direct integration with the university SIS (Banner). Students view live academic data without visiting the registrar.

- GPA display card with current semester and cumulative GPA
- Predicted GPA meter (AI-powered, shows probability range)
- Course schedule: weekly timetable view (Mon-Sun, color-coded per subject)
- Attendance tracker: percentage per course + warning if below threshold
- Exam calendar: upcoming exams with countdown timers
- Course registration: available courses list with pre-requisite check
- Transcript download (PDF generation on-demand)
- Red alert if attendance < 75% for any course

Screen 6 Payment Screen — Chat-to-Pay Integration

Native in-app payment without redirecting to an external website. Students pay tuition, library fines, or lab fees directly through EDYGUIDE using Fawry or Vodafone Cash.

- Outstanding balance display: 'Tuition Fee: EGP 5,000'
- Payment method selection: Fawry | Credit/Debit Card | Vodafone Cash
- Fawry: generates a reference number for kiosk payment
- Vodafone Cash: direct mobile wallet debit via EasyKash/Paymob API
- Payment history: list of past transactions with status badges
- Receipt generation (PDF) after successful payment
- Fine tracker: library fines, lab equipment damage costs
- Document fee payment: 'To Whom It May Concern' certificate fee
- Document tracking status: 'Signed by Dean — currently at Registrar'

Screen 7 Career Bridge — Internship & CV Screen

Transforms EDYGUIDE into a personal career coach. Uses Llama-3 to analyze the student's transcript and match them to real Egyptian job market opportunities.

- CV Upload section with 'Scan for ATS' button
- ATS Score meter (0-100) with improvement suggestions
- CV feedback in Arabic or English based on Egyptian startup standards
- Suggested Internships cards: company name, role, deadline, 'Apply Now' button
- Internship data sourced from Wuzzuf/LinkedIn API scan
- Skill Gap Analysis: 'Most Data Analyst jobs need PowerBI — you haven't taken it yet'
- Free certification recommendations based on missing skills
- Alumni Matchmaker: 'Connect with an alumnus at Valeo?' — schedules coffee chat
- Mock Interview Simulator: text-based practice with AI feedback
- Elevator Pitch Generator: 30-second personal intro based on transcript

Screen 8 Wellness Hub — Mental Health Support Screen

A confidential, Arabic-language mental health support feature. Culturally sensitive and privacy-protected under Law 151/2020.

- Emoji-based mood tracker: 5-point scale (Very Stressed → Great)
- Daily/weekly mood trend graph (anonymous, stored encrypted)
- 'Talk to Saleem' button: launches Arabic mental health chatbot
- Crisis detection: if distress language detected, warm handoff to live counselor
- Guided mindfulness exercises: 5-minute audio sessions (midterms/finals)
- Resource library: localized Arabic videos on managing stress and anxiety
- Anonymous mode: all wellness data is NOT linked to student academic record
- Faculty alert (aggregated only): '30% of Engineering students are stressed this week'
- 24/7 hotline quick-dial button

Screen 9

Progress Tracking — Proactive AI Nudging System

The SVM model monitors student data in the background and automatically sends alerts before problems become critical.

- Progress dashboard: attendance %, assignment completion, GPA trend
- Risk indicator: Green (Safe) / Yellow (Watch) / Red (At-Risk)
- Push notification examples:
 - 'You have a 40% assignment due in 48 hours — start now?'
 - 'Your attendance in Physics is at 71% — attend next 2 classes to stay above limit'
 - 'Based on current pace, you can graduate one semester early if you add this course'
- Predictive graduation tracker: semester-by-semester projection
- Study group matchmaker: finds students in same course with similar free time
- Notification preferences: student can control frequency and type of nudges

Chapter 2: Technology Stack — Full Explanation & Core Code

Every technology used in EDYGUIDE is explained below with: (1) What it is, (2) Why it was chosen, (3) Its advantages and limitations, and (4) The main code patterns used in the project.

2.1 Flutter (Frontend Framework)

Flutter

An open-source UI toolkit by Google for building natively compiled apps for mobile, web, and desktop from a single codebase using the Dart language.

Advantages

- ✓ One codebase → iOS + Android + Web simultaneously
- ✓ Hot reload: see UI changes instantly during development
- ✓ Rich widget library: pre-built Material Design components
- ✓ High performance: compiles to native ARM code (no JavaScript bridge)
- ✓ Strong Google/community support and growing in Egypt

Limitations

- ✗ Dart language is less common than JavaScript/Python
- ✗ Large app bundle size compared to native apps
- ✗ Some platform-specific features require native plugins
- ✗ Older iOS versions may have compatibility issues

Main Flutter Code — Home Dashboard Screen

```
// main.dart — EDYGUIDE App Entry Point
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp();
  runApp(const EdyGuideApp());
}

class EdyGuideApp extends StatelessWidget {
  const EdyGuideApp({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'EDYGUIDE',
      theme: ThemeData(primaryColor: Color(0xFF1A3A6B)),
      home: const HomeScreen(),
    );
  }
}
```

```
);  
}  
}
```

```
// home_screen.dart – Dashboard with Daily Nudge  
class HomeScreen extends StatelessWidget {  
  final String studentName = 'Ahmed';  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        backgroundColor: Color(0xFF1A3A6B),  
        title: Image.asset('assets/edyguide_logo.png', height: 36),  
      ),  
      body: SingleChildScrollView(  
        padding: EdgeInsets.all(16),  
        child: Column(children: [  
          // Personalized greeting  
          Text('Welcome $studentName',  
            style: TextStyle(fontSize: 24, fontWeight: FontWeight.bold)),  
          SizedBox(height: 12),  
          // Daily nudge card  
          NudgeCard(message: 'You have Physics at 10 AM – don\'t miss this one!'),  
          SizedBox(height: 16),  
          // Service grid  
          ServiceGrid(),  
        ]),  
      ),  
      bottomNavigationBar: BottomNavBar(),  
    );  
  }  
}
```

```
// service_grid.dart – Tappable service cards  
class ServiceGrid extends StatelessWidget {  
  final List services = [  
    {'icon': Icons.chat_bubble, 'label': 'Chat', 'route': '/chat'},  
    {'icon': Icons.calendar_today, 'label': 'Schedule', 'route': '/schedule'},  
    {'icon': Icons.assignment, 'label': 'Exams', 'route': '/exams'},  
    {'icon': Icons.payment, 'label': 'Pay Tuition', 'route': '/payment'},  
    {'icon': Icons.school, 'label': 'View GPA', 'route': '/gpa'},  
    {'icon': Icons.favorite, 'label': 'Mental Health', 'route': '/wellness'},  
    {'icon': Icons.work, 'label': 'Career', 'route': '/career'},  
    {'icon': Icons.bar_chart, 'label': 'Attendance', 'route': '/attendance'},  
  ];  
  @override
```



```
Widget build(BuildContext context) {  
  return GridView.builder(  
    shrinkWrap: true,  
    gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(  
      crossAxisCount: 4, mainAxisSpacing: 12, crossAxisSpacing: 12),  
    itemCount: services.length,  
    itemBuilder: (ctx, i) => ServiceCard(data: services[i]),  
  );  
}
```

2.2 FastAPI (Python Backend)

FastAPI

A modern, fast Python web framework for building APIs with automatic documentation. Built on Starlette and Pydantic, designed for async operations.

Advantages

- ✓ Extremely fast: one of the fastest Python frameworks (rivals Node.js)
- ✓ Auto-generates Swagger API documentation
- ✓ Native async/await support: handles many simultaneous AI requests
- ✓ Type hints with Pydantic: catches errors before runtime
- ✓ Easy integration with ML models, Firebase, and payment SDKs

Limitations

- ✗ Less 'batteries included' than Django (no built-in admin panel)
- ✗ Smaller community than Flask/Django
- ✗ Requires understanding of async programming
- ✗ No built-in ORM for relational databases

Main FastAPI Code — Backend API Routes

```
# main.py — FastAPI Application Entry
from fastapi import FastAPI, Depends, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from routers import chat, academic, payment, career, wellness

app = FastAPI(title='EDYGUIDE API', version='1.0.0')
app.add_middleware(CORSMiddleware,
    allow_origins=['*'], allow_methods=['*'], allow_headers=['*'])

# Register all route modules
app.include_router(chat.router, prefix='/api/chat', tags=['Chat'])
app.include_router(academic.router, prefix='/api/academic', tags=['Academic'])
app.include_router(payment.router, prefix='/api/payment', tags=['Payment'])
app.include_router(career.router, prefix='/api/career', tags=['Career'])
app.include_router(wellness.router, prefix='/api/wellness', tags=['Wellness'])

@app.get('/')
async def root(): return {'status': 'EDYGUIDE API Running'}
```

```
# routers/chat.py — AI Chatbot Endpoint
from fastapi import APIRouter, Body
from services.llm_service import get_llm_response
from services.firebase_service import save_chat_message

router = APIRouter()

class ChatRequest(BaseModel):
    student_id: str
    session_id: str
    message: str
    language: str = 'ar' # 'ar' or 'en'
```

```
@router.post('/send')
async def send_message(req: ChatRequest):
    # Save student message to Firebase
    await save_chat_message(req.session_id, req.message, 'student')
    # Get AI response from Llama-3
    ai_reply = await get_llm_response(
        message=req.message,
        student_id=req.student_id,
        language=req.language
    )
    # Save AI response
    await save_chat_message(req.session_id, ai_reply, 'bot')
    return {'reply': ai_reply, 'session_id': req.session_id}
```

```
# services/llm_service.py – Llama-3 Integration
import httpx

WATSONX_URL = 'https://us-south.ml.cloud.ibm.com/ml/v1/text/generation'
WATSONX_TOKEN = 'YOUR_IBM_TOKEN'

async def get_llm_response(message: str, student_id: str, language: str):
    system_prompt = '''You are EDYGUIDE, a helpful Egyptian university assistant.
    Always respond in {language}. Be concise and student-friendly.
    You have access to the student's grades, schedule, and attendance.'''

    async with httpx.AsyncClient() as client:
        response = await client.post(WATSONX_URL,
            headers={'Authorization': f'Bearer {WATSONX_TOKEN}'},
            json={
                'model_id': 'meta-llama/llama-3-70b-instruct',
                'input': f'{system_prompt}\nStudent: {message}\nEDYGUIDE:',
                'parameters': {'max_new_tokens': 300, 'temperature': 0.7}
            })
    return response.json()['results'][0]['generated_text']
```

2.3 Firebase Firestore (Database)

Firebase	Google's cloud-based platform providing a real-time NoSQL database (Firestore), authentication, and hosting. Stores all EDYGUIDE data with end-to-end encryption.
Advantages	Limitations
✓ Real-time sync: chat messages appear instantly without page refresh ✓ No SQL knowledge needed — JSON-like document model ✓ Built-in Authentication (email, SSO, phone OTP) ✓ Offline support: app works without internet, syncs when reconnected ✓ Scales automatically — handles 20,000+ concurrent users ✓ Free tier sufficient for prototype/testing phase	✗ NoSQL limits complex relational queries ✗ Costs can grow rapidly at large scale ✗ Data stored on Google servers (jurisdiction concern for Law 151) ✗ No built-in full-text search (requires Algolia/ElasticSearch add-on)

Main Firebase Code — Database Operations

```
# services/firebase_service.py - Firestore Operations

import firebase_admin
from firebase_admin import credentials, firestore
from datetime import datetime

# Initialize Firebase
cred = credentials.Certificate('serviceAccountKey.json')
firebase_admin.initialize_app(cred)
db = firestore.client()

async def save_chat_message(session_id: str, text: str, sender: str):
    '''Save a chat message to Firestore.'''
    db.collection('chat_sessions').document(session_id) \
        .collection('messages').add({
        'text': text,
        'sender': sender,          # 'student' or 'bot'
        'sent_at': datetime.now(),
        'read': False
    })

async def get_student_profile(student_id: str):
    '''Fetch student data from Firestore.'''
    doc = db.collection('students').document(student_id).get()
    if doc.exists:
        return doc.to_dict()
    raise ValueError(f'Student {student_id} not found')

async def update_attendance(student_id: str, course_id: str, attended: bool):
```

```
'''Update attendance record.'''
ref = db.collection('students').document(student_id) \
    .collection('attendance').document(course_id)
ref.set({'attended': attended, 'updated_at': datetime.now()}, merge=True)
```

```
// Flutter: Firebase real-time chat listener (chat_screen.dart)
Stream getChatMessages(String sessionId) {
  return FirebaseFirestore.instance
    .collection('chat_sessions')
    .doc(sessionId)
    .collection('messages')
    .orderBy('sent_at', descending: false)
    .snapshots(); // Real-time updates
}

// Use in Widget:
StreamBuilder(
  stream: getChatMessages(sessionId),
  builder: (context, snapshot) {
    final messages = snapshot.data?.docs ?? [];
    return ListView.builder(
      itemCount: messages.length,
      itemBuilder: (ctx, i) => ChatBubble(
        text: messages[i]['text'],
        isBot: messages[i]['sender'] == 'bot',
      ),
    );
  },
)
```

2.4 Llama-3 — Large Language Model (AI Engine)

Llama-3 (LLM)

Meta's open-source large language model with 70 billion parameters. Hosted via IBM Watsonx.ai for EDYGUIDE's conversational AI, CV analysis, and career coaching features.

Advantages

- ✓ Open-source: no per-token licensing fee (unlike GPT-4)
- ✓ Supports Arabic language (MSA and some dialects)
- ✓ Can be fine-tuned on university-specific data
- ✓ Strong reasoning for GPA prediction explanations
- ✓ IBM Watsonx hosting provides enterprise-grade security

Limitations

- ✗ Requires GPU infrastructure (expensive to self-host)
- ✗ Egyptian dialect Arabic support is limited (needs fine-tuning)
- ✗ Response latency: 2-5 seconds per request
- ✗ Hallucinations possible — must validate against real SIS data
- ✗ API costs grow with usage volume

Main AI Code — CV Analysis & GPA Prediction

```
# services/cv_analyzer.py - Resume Analysis with Llama-3
async def analyze_cv(cv_text: str, student_gpa: float, target_role: str):
    prompt = f'''
    You are a career advisor for Egyptian university students.
    Analyze this CV for a student with GPA {student_gpa} applying for {target_role}.
    CV Content: {cv_text[:2000]}

    Provide feedback in JSON format:
    {{
      'ats_score': <0-100>,
      'strengths': [],
      'improvements': [],
      'missing_skills': [],
      'overall_advice': '<2 sentences in Arabic>'
    }}
    '''

    response = await get_llm_response(prompt, language='ar')
    import json
    return json.loads(response)

# services/gpa_predictor.py - Conversational GPA Prediction
async def predict_gpa(student_id: str, hypothetical_grades: dict):
    student = await get_student_profile(student_id)
    current_gpa = student['gpa']
    completed_hours = student['credit_hours_completed']

    # Calculate predicted GPA
    new_points = sum(
        grade_to_points(g) * hours
        for course, (g, hours) in hypothetical_grades.items()
```

```
)  
total_hours = completed_hours + sum(h for _, h in hypothetical_grades.values())  
predicted = (current_gpa * completed_hours + new_points) / total_hours  
return round(predicted, 2)
```

2.5 Support Vector Machine (SVM) — Predictive Analytics

SVM Model

A supervised machine learning algorithm used in EDYGUIDE to predict student academic risk. Trained on historical student data (attendance, grades, assignment completion) to identify at-risk students proactively.

Advantages

- ✓ Effective with small-to-medium datasets (university data size)
- ✓ Works well for binary classification (at-risk vs safe)
- ✓ Resistant to overfitting with proper kernel choice
- ✓ Interpretable: can explain why a student is flagged as at-risk
- ✓ Scikit-learn implementation is production-ready and well-documented

Limitations

- ✗ Slow to train on very large datasets
- ✗ Feature engineering requires domain expertise
- ✗ Needs periodic retraining as student behavior patterns change
- ✗ Sensitive to feature scaling — normalization required

Main SVM Code — At-Risk Student Detection

```
# ml/risk_predictor.py - SVM Student Risk Model

from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
import joblib
import numpy as np

# Feature vector for each student:
# [attendance_pct, avg_grade, assignments_completed_pct,
#  days_since_last_login, gpa_trend (this_sem - last_sem)]

def train_risk_model(X_train, y_train):
    '''Train SVM on historical student data.'''
    model = Pipeline([
        ('scaler', StandardScaler()),
        ('svm', SVC(kernel='rbf', probability=True, C=1.0))
    ])
    model.fit(X_train, y_train)
    joblib.dump(model, 'models/risk_model.pkl')
    return model

def predict_risk(student_features: dict) -> dict:
    '''Predict if student is at academic risk.'''
    model = joblib.load('models/risk_model.pkl')
    features = np.array([
        student_features['attendance_pct'],
        student_features['avg_grade'],
        student_features['assignments_pct'],
        student_features['days_since_login'],
```



```
        student_features['gpa_trend']
    ]])
    prob = model.predict_proba(features)[0]
    risk_probability = prob[1] # Probability of being 'at-risk'
    return {
        'risk_probability': round(risk_probability, 3),
        'is_at_risk': risk_probability > 0.6,
        'nudge_required': risk_probability > 0.4
    }
```

Automated nudge trigger (runs every 6 hours via CRON)

```
async def check_all_students_and_nudge():
    students = await get_all_active_students()
    for student in students:
        features = await build_feature_vector(student['id'])
        result = predict_risk(features)
        if result['nudge_required']:
            await send_push_notification(
                student_id=student['id'],
                message=generate_nudge_message(features, result)
            )
```

2.6 Fawry & Vodafone Cash (Payment Integration)

Fawry SDK	Egypt's leading payment gateway allowing students to pay through Fawry kiosks, credit cards, or mobile wallets. Integrated natively into EDYGUIDE for in-app tuition and fee payments.
Advantages	Limitations
✓ Widest network in Egypt: 300,000+ Fawry kiosks ✓ Supports unbanked students (kiosk cash payment) ✓ Simple SDK: generates reference number for offline payment ✓ Vodafone Cash: instant mobile wallet deduction ✓ PCI-DSS compliant: secure card data handling	✗ Fawry kiosk payment not instant — 24hr confirmation delay ✗ Integration requires merchant account approval (takes time) ✗ Vodafone Cash only for Vodafone subscribers ✗ Transaction fees: typically 1-2.5% per transaction

Main Payment Code — Fawry Integration

```
# services/payment_service.py – Fawry Payment Integration
import hashlib, httpx
from datetime import datetime

FAWRY_BASE_URL = 'https://atfawry.fawrystaging.com/ECommerceWeb/Fawry/payments'
MERCHANT_CODE = 'YOUR_MERCHANT_CODE'
SECURITY_KEY = 'YOUR_SECURITY_KEY'

def generate_fawry_signature(merchant_code, merchant_ref, customer_id,
                             amount, security_key):
    '''Generate SHA-256 signature for Fawry API authentication.'''
    raw = f'{merchant_code}{merchant_ref}{customer_id}{amount:.2f}{security_key}'
    return hashlib.sha256(raw.encode()).hexdigest()

async def create_fawry_payment(student_id: str, amount: float,
                                description: str) -> dict:
    merchant_ref = f'EDY-{student_id}-{int(datetime.now().timestamp())}'
    signature = generate_fawry_signature(
        MERCHANT_CODE, merchant_ref, student_id, amount, SECURITY_KEY)

    payload = {
        'merchantCode': MERCHANT_CODE,
        'merchantRefNum': merchant_ref,
        'customerProfileId': student_id,
        'paymentMethod': 'PAYATFAWRY',
        'amount': amount,
        'currencyCode': 'EGP',
        'description': description,
        'signature': signature
    }

    async with httpx.AsyncClient() as client:
```

```
        resp = await client.post(f'{FAWRY_BASE_URL}/charge', json=payload)
        result = resp.json()
        return {
            'reference_number': result.get('referenceNumber'),
            'status': result.get('statusDescription'),
            'amount': amount
        }

    @router.post('/api/payment/fawry')
    async def pay_with_fawry(student_id: str, amount: float, description: str):
        result = await create_fawry_payment(student_id, amount, description)
        await save_payment_record(student_id, result)
        return result
```

2.7 Authentication & Security (JWT + Law 151/2020)

Security	EDYGUIDE uses JWT tokens for stateless authentication, MFA via OTP, SSO with the university's Banner/LDAP system, and full compliance with Egypt's Personal Data Protection Law No. 151 of 2020.
Advantages	Limitations
✓ JWT: stateless — no session storage needed on server ✓ MFA: significantly reduces unauthorized access risk ✓ SSO: students use existing university credentials (no new password) ✓ Law 151 compliance: builds student trust for mental health data ✓ End-to-end encryption: all sensitive data encrypted at rest and in transit	✗ JWT tokens cannot be invalidated before expiry (use refresh tokens) ✗ SSO requires IT department cooperation to set up LDAP access ✗ Law 151 compliance requires Data Protection Officer (DPO) appointment ✗ MFA adds friction — some students may find it annoying

Main Security Code — JWT Authentication

```
# services/auth_service.py — JWT Authentication

from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from jose import JWTError, jwt
from datetime import datetime, timedelta
import pyotp

SECRET_KEY = 'your-256-bit-secret-stored-in-env-variable'
ALGORITHM = 'HS256'
ACCESS_TOKEN_EXPIRE_MINUTES = 30

def create_access_token(student_id: str) -> str:
    '''Issue JWT token after successful login.'''
    expire = datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    payload = {'sub': student_id, 'exp': expire}
    return jwt.encode(payload, SECRET_KEY, algorithm=ALGORITHM)

def verify_token(token: str) -> str:
    '''Verify JWT and return student_id.'''
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        return payload.get('sub')
    except JWTError:
        raise HTTPException(status_code=401, detail='Invalid or expired token')

def generate_otp(student_id: str) -> str:
    '''Generate TOTP for MFA.'''
    totp = pyotp.TOTP(pyotp.random_base32())
    return totp.now() # 6-digit code, valid 30 seconds
```

```
@router.post('/api/auth/login')
async def login(student_id: str, password: str):
    student = await verify_university_credentials(student_id, password)
    if not student:
        raise HTTPException(status_code=401, detail='Invalid credentials')
    otp = generate_otp(student_id)
    await send_otp_email(student['email'], otp) # MFA step
    return {'message': 'OTP sent to university email', 'student_id': student_id}

@router.post('/api/auth/verify-otp')
async def verify_otp(student_id: str, otp: str):
    if await check_otp(student_id, otp):
        token = create_access_token(student_id)
        return {'access_token': token, 'token_type': 'bearer'}
    raise HTTPException(status_code=401, detail='Invalid OTP')
```

Chapter 3: Database Schema & Data Model

EDYGUIDE uses Firebase Firestore (NoSQL) as its primary database. Data is organized in collections and sub-collections. Below is the complete schema for all entities.

```
// Firestore Database Structure

// Collection: students
students/{student_id} {
  student_name:  string,
  email:         string,
  phone:         string,
  department:    string,
  faculty:       string,
  academic_year: string,
  gpa:           number,
  credit_hours:  number,
  created_at:    timestamp
}

// Sub-collection: students/{id}/attendance
attendance/{course_id} {
  course_name:  string,
  total_lectures: number,
  attended:     number,
  percentage:   number,    // auto-calculated
  warning_sent: boolean
}

// Collection: chat_sessions
chat_sessions/{session_id} {
  student_id:  string,
  start_time:  timestamp,
  end_time:    timestamp,
  language:    string    // 'ar' or 'en'
}

// Sub-collection: chat_sessions/{id}/messages
messages/{message_id} {
  text:        string,
  sender:      string,  // 'student' or 'bot'
  sent_at:     timestamp,
  read:        boolean
}

// Collection: payments
payments/{transaction_id} {
```

```
    student_id:    string,
    amount:        number,
    currency:       string,    // 'EGP'
    method:         string,    // 'FAWRY' | 'VODAFONE' | 'CARD'
    status:         string,    // 'pending' | 'paid' | 'failed'
    fawry_ref:      string,
    description:    string,
    created_at:     timestamp
  }

// Collection: wellness_records (ENCRYPTED, anonymous)
wellness_records/{record_id} {
  anonymous_hash: string,    // NOT student_id – privacy protected
  mood_score:    number,    // 1-5
  stress_level:   string,
  notes:         string,    // encrypted
  escalated:     boolean,
  created_at:    timestamp
}
```

Chapter 4: Project Fixes & Recommendations

Based on the detailed analysis of the original project document, the following fixes are required before academic submission.

	Name Inconsistency
Critical	The document uses EDYGUIDE, EDUGUIDE, CampusGrow Bot, and AISSE interchangeably. Choose one name (EDYGUIDE) and replace all occurrences throughout the document.
	Foreign Literature Survey Section
Critical	Pages 8-9 describe 'CampusGrow Bot' — a completely different project. This section must be rewritten to analyze systems directly relevant to EDYGUIDE: AUC Saleem, MUSTER, myGUC, and AASTMT Portal.
	Unsupported Statistics in Abstract
Critical	The Abstract claims '21% increase in engagement' as a result. This is a proposed system with no deployed users. Change to 'projected 21% increase' and add the citation source.
	Missing References Chapter
High	The Table of Contents lists References, but no references appear in the document. All statistics (85% mental health stat, 7% retention improvement) need proper citations.
	ERD Mismatch
High	The written ERD entities (Student, Course, Payment, Career_Profile) do not match the diagram which shows feedback, faq, admins, services tables. Synchronize both.
	Missing UI Screens
High	The interface mockup only shows the Dashboard and Chatbot. Add screens for: Wellness Hub (mood tracker), Career Bridge (CV scanner + ATS score), and the Concierge GPA prediction chat.
	Shallow Methodology
Medium	The methodology lists technologies but does not explain HOW they work together. Add data flow diagrams showing: student request → FastAPI → Firebase → Llama-3 → response.
	Mental Health Feature Under-represented
Medium	Mental Health Support is described as a key differentiator but shown as a single small icon on the dashboard. It needs its own dedicated screen with the full Saleem chatbot, mood tracker, and crisis escalation flow.

	No Use Case Diagrams
Low	The document references a DFD (Level 0) but does not include UML use case diagrams, sequence diagrams, or activity diagrams. These are expected in a formal IT project report.
	Law 151 Details Vague
Low	Compliance with Law 151/2020 is mentioned multiple times but never explained. Add a dedicated section explaining what data is collected, consent mechanism, data retention policy, and the role of the DPO.

Chapter 5: Implementation Roadmap & Phase Plan

Phase	Timeline	Deliverables	Tech Focus
Phase 1 Foundation	Months 1–2	SIS/LMS API access setup Law 151 compliance framework Core dev team hired Firebase project initialized	Firebase, FastAPI setup, LDAP/SSO integration
Phase 2 Core Services	Months 3–5	Flutter mobile shell complete Fawry & Vodafone Cash integrated Grades/schedule/attendance live	Flutter, Fawry SDK, Firebase Firestore
Phase 3 AI Concierge	Months 6–8	Llama-3 chatbot deployed SVM risk model trained GPA prediction feature live Proactive nudging active	Llama-3 (Watsonx), Scikit-learn SVM, Push notific
Phase 4 Wellness & Career	Months 9–11	Arabic Saleem mental health bot CV ATS scanner live Career Bridge with Wuzzuf API Mock interview simulator	Arabic NLP, Llama-3 CV analysis, Wuzzuf/Linked
Phase 5 Launch & Scale	Month 12+	Beta: 500 student pilot Full campus deployment Iterate on engagement metrics Blockchain certificate pilot	Load testing, AWS EC2 scaling, analytics dashbo

Project Summary: EDYGUIDE is a comprehensive AI-powered student service ecosystem specifically designed for Egyptian universities. It uniquely combines proactive academic support (SVM-based nudging), confidential Arabic mental health assistance (Saleem model), native payment integration (Fawry/Vodafone Cash), and LLM-powered career coaching (Llama-3 CV analysis + Wuzzuf job matching). Built on Flutter + FastAPI + Firebase, it is fully compliant with Egypt's Law 151/2020 and designed to scale to 20,000+ concurrent users. The projected outcome is a 21% increase in student service engagement and a measurable improvement in retention rates through early at-risk identification.